

STRUCTURE PADDING IN C

1. What is Structure Padding?

Structure padding refers to the extra unused bytes inserted by the compiler between structure members or at the end of a structure. This is done to satisfy **alignment requirements** imposed by the CPU architecture.

Modern processors read memory in blocks (words). If data is misaligned, the CPU may need multiple memory accesses, making the program slower. Padding helps avoid this performance penalty.

2. What is Alignment?

Alignment means placing data at memory addresses that are multiples of a specific number. Each data type has a natural alignment requirement.

Rule: A variable must start at an address that is a multiple of its alignment.

Data Type	Size (bytes)	Alignment (bytes)
char	1	1
short	2	2
int	4	4
float	4	4
double	8	8
pointer (64-bit)	8	8

3. Core Rules of Structure Padding

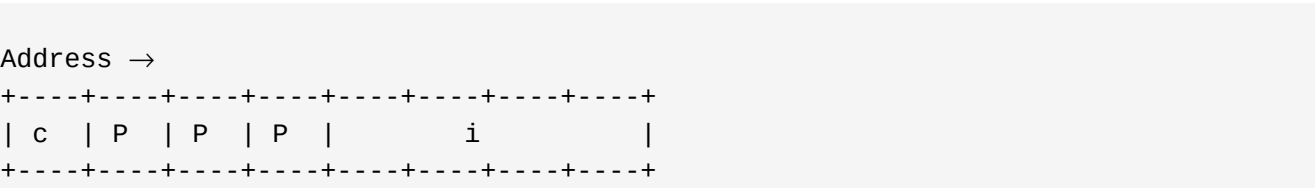
- Each structure member is placed at an offset that satisfies its alignment requirement.
- If the current offset is not suitable, padding bytes are inserted.
- Padding depends on the **order of members**.
- The total size of the structure is a multiple of the largest alignment among its members.
- Padding at the end ensures correct alignment when structures are stored in arrays.

4. Example 1: Basic Padding

Consider the following structure:

```
struct A { char c; int i; };
```

The character requires 1-byte alignment. The integer requires 4-byte alignment. After placing **char c**, the next available offset is 1, which is not suitable for **int i**. Therefore, 3 bytes of padding are added.

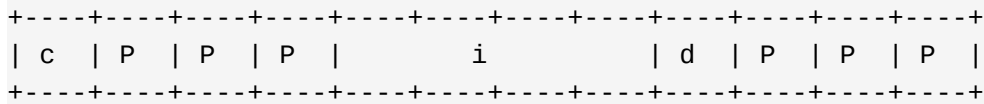


sizeof(struct A) = 8 bytes

5. Example 2: Padding at the End

```
struct B { char c; int i; char d; };
```

After placing **char d**, the total size becomes 9 bytes. However, the largest alignment required is 4, so the compiler pads the structure to 12 bytes.



sizeof(struct B) = 12 bytes

6. Tricky Example (Exam Favorite)

```
struct C { char a; double d; char b; };
```

This example often confuses students because **double** has 8-byte alignment. After placing **char a**, 7 bytes of padding are needed before **double d**. After placing **char b**, the structure must again be padded to a multiple of 8.

Calculation: 1 (a) + 7 (pad) + 8 (double) + 1 (b) + 7 (final pad) = 24 bytes

7. How to Reduce Padding

Reordering structure members from largest to smallest alignment can significantly reduce padding and memory usage.

Bad Order:

```
struct Bad { char c; double d; int i; };
```

Good Order:

```
struct Good { double d; int i; char c; };
```

8. Packing Structures (Advanced)

Compilers provide directives like **#pragma pack** or **__attribute__((packed))** to disable padding. While this reduces size, it may cause performance issues or hardware faults on some architectures.

9. Exam-Oriented Summary

- Padding is added for alignment, not for memory saving.
- Member order affects structure size.
- Largest alignment decides final structure size.
- Always calculate offsets step-by-step.